

Lab5 GPU 编程实验报告

并行程序设计

2026 年 6 月 18 日

1 实验目的与任务说明

本次实验包括基础要求和进阶选题两部分。基础部分总结 AMD AUP 平台 HIP Programming Course 中 6 个 Notebook 的学习过程，覆盖 HIP kernel、线程索引、显存管理、原子操作、shared memory、归约和迭代式 GPU 算法。进阶部分选择 PCFG 口令猜测选题，将 PT 内部批量候选口令生成迁移到 AMD HIP GPU kernel 中，并在单 PT baseline 之外实现多 PT 一次装载、CPU/GPU 重叠和按 PT 工作量调度三个分支。

本报告为跑正式数据前的报告草稿。性能实验结果只保留 TODO 占位，不填写任何未运行的性能数据。

2 AMD AUP 平台 HIP 课程学习

2.1 Lab1 Vector Addition

向量加法中每个线程负责一个元素，核心是计算一维全局线程下标并做边界检查。

```
__global__ void vector_add(const float* A, const float* B, float* C, int N) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    if (idx < N) {  
        C[idx] = A[idx] + B[idx];  
    }  
}
```

`blockIdx.x * blockDim.x + threadIdx.x` 将 block 内局部线程号转换为全局编号。由于 grid size 通常向上取整，总线程数可能大于元素数，因此必须使用 `idx < N` 防止越界。host 端通过 `hipMalloc` 分配 device 显存，使用 `hipMemcpy` 传输输入和输出，kernel launch 启动 GPU 计算，最后用 `hipDeviceSynchronize` 等待 GPU 完成。

学习过程中通过线程编号例子理解：当 `blockIdx.x=0`, `blockDim.x=256`, `threadIdx.x=100` 时，全局下标为 100；当 `blockIdx.x=2`, `blockDim.x=128`, `threadIdx.x=50` 时，全局下标为 306；当 `blockIdx.x=5`, `blockDim.x=64`,

threadIdx.x=0 时，全局下标为 320。若 N=1000，block size 为 64、128、256、512 时，grid size 分别为 16、8、4、2，总线程数均为 1024，空闲线程均为 24。多元素每线程版本中，stride 应为 blockDim.x * gridDim.x。

2.2 Lab2 Matrix Transpose

矩阵转置使用二维 grid/block 更自然。row-major 线性内存中，输入位置为 row * width + col，转置后的输出位置为 col * height + row。

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;

if (row < height && col < width) {
    output[col * height + row] = input[row * width + col];
}
```

直接转置通常读访问连续、写访问跨 stride，容易出现非合并访存。tilted transpose 的思路是先把 tile 读入 shared memory，再交换 block 方向写出，从而改善 global memory 访问模式。shared memory 声明为 tile[BLOCK_SIZE][BLOCK_SIZE + 1] 时，额外一列可减少 bank conflict。

当 blockDim=(32,32) 时，blockIdx=(0,0)，threadIdx=(5,10) 的全局下标为 (5,10)；blockIdx=(1,0)，threadIdx=(0,0) 的全局下标为 (32,0)；blockIdx=(2,3)，threadIdx=(15,20) 的全局下标为 (79,116)。对 rows=100，cols=200 的矩阵，如果 block 为 32 × 32，需要 7 × 4 个 blocks。

4×6 矩阵中，线程 (1,0) 的 input index 为 1、output index 为 4；线程 (0,1) 的 input index 为 6、output index 为 1；线程 (3,2) 的 input index 为 15、output index 为 14。若 BLOCK_SIZE=32 且使用 float，tile[32][33] 占用 32 × 33 × 4 = 4224 bytes shared memory。

2.3 Lab3 Histogram

Histogram 的关键问题是多个线程会同时更新同一个 bin。基础并行版本需要原子操作保证正确性。

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < n) {
    int bin = input[idx];
    atomicAdd(&hist[bin], 1);
}
```

全局内存上的 atomicAdd 在输入分布高度集中时会发生严重冲突，性能可能被串行化。优化方向是在 block 内使用 shared memory 构建局部 histogram，最后再把每个 block

的局部统计合并到 global memory。这样每个输入元素只需从 global memory 读取一次，相比每个 bin 扫描全数组的 naive 方法显著减少内存流量。

naive 版本中每个 block 负责一个 bin，并扫描整个输入，所以每个输入元素会被读取 `num_bins` 次。当 `num_bins=256`，`N=10M` 时，总读取次数为 2.56B；若输入为 `int`，仅输入读取约为 10.24GB。优化版本每个输入元素只读一次，理论读取次数下降比例为 `num_bins`，在 256 bins 时为 256 倍。shared memory atomic 通常比 global memory atomic 快，因为访问延迟更低且作用域限制在 block 内。

2.4 Lab4 Reduction

Reduction 把多个元素合并为一个结果。shared memory tree reduction 每轮将参与线程数减半。

```
extern __shared__ float sdata[];

int tid = threadIdx.x;
int i = blockIdx.x * blockDim.x + threadIdx.x;

sdata[tid] = (i < n) ? input[i] : 0.0f;
__syncthreads();

for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
    if (tid < stride) {
        sdata[tid] += sdata[tid + stride];
    }
    __syncthreads();
}

if (tid == 0) {
    partial[blockIdx.x] = sdata[0];
}
```

`__syncthreads()` 保证同一 block 内所有线程完成当前 stride 的读写后再进入下一轮。block 之间不能用 `__syncthreads()` 同步，因此多 block reduction 需要输出 partial result，再通过第二轮 kernel 或 CPU 汇总。相比 naive global atomic，tree reduction 减少了全局原子操作数量。

1024 个元素、block size 256 时，每个 block 内需要 $\log_2 256 = 8$ 个 reduction steps。block 级 reduction 后剩余 $\lceil 1024/256 \rceil = 4$ 个 partial sums。若 $N = 1,000,000$ ，block size 64 会产生 15625 个 partial 或最终 atomic，block size 1024 会产生 977 个。tree reduction 第一轮之后约一半线程进入空闲。

2.5 Lab5 Monte Carlo Integration

Monte Carlo 积分通过大量采样估计积分值。每个采样点之间相互独立，适合让 GPU 线程并行处理。

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
double local = 0.0;

for (int i = idx; i < n_samples; i += blockDim.x * gridDim.x) {
    double x = sample_x(i);
    local += f(x);
}

atomicAdd(result, local);
```

最终需要对所有线程或 block 的局部结果做归约。性能瓶颈主要可能来自随机数生成、函数计算和归约过程。shared memory reduction 可以把大量 global atomic 降为每个 block 一次 global atomic，从而降低冲突。

atomic reduction 方法逻辑工作量为 $O(n_samples)$ ，但高冲突 atomic 可能导致串行化。若使用 256 threads/block 且 $N = 1M$ ，每个 block 输出一 partial 时，global atomic 数约为 $\lceil 1,000,000/256 \rceil = 3907$ ；若固定 256 个 blocks，则 global atomic 为 256 次。double precision 通常提供约 15 到 16 位十进制有效数字；若换成 float，超大样本数累加时更容易出现舍入误差。

2.6 Lab6 K-Means Clustering

K-Means 的 assignment 阶段天然并行，每个线程处理一个数据点并寻找最近中心。

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < num_points) {
    int best_cluster = 0;
    float best_dist = distance(points[idx], centroids[0]);

    for (int c = 1; c < k; c++) {
        float d = distance(points[idx], centroids[c]);
        if (d < best_dist) {
            best_dist = d;
            best_cluster = c;
        }
    }

    labels[idx] = best_cluster;
}
```

```
}

```

分配阶段每个点独立计算，单线程复杂度为 $O(k)$ 。当 $k = 100$ 时，每个线程执行 100 次距离比较。更新中心阶段需要把同一 cluster 的点坐标求和并统计数量，存在写冲突，可用 `atomicAdd` 或 block 内归约降低全局冲突。K-Means 是迭代算法，每一轮中心依赖上一轮输出，因此轮与轮之间需要 kernel 结束后的全局同步。

对 $N = 50,000, k = 50$ ，assignment 阶段每点 50 次距离计算，总计 2,500,000 次距离计算。若更新阶段每个点对 `x_sum`、`y_sum`、`count` 做 3 次 shared atomic，总 shared atomic 为 150,000 次。256 threads/block 时 block 数为 196，若每 block 对每 cluster 写 3 个统计量，global atomic 约为 $196 \times 50 \times 3 = 29,400$ 次。

3 口令猜测选题的 GPU 并行化设计

3.1 原串行框架分析

本次 PCFG 口令猜测源码位于 `pcfg_gpu/guess_mpi_advanced/`。核心文件包括 `PCFG.h`、`guessing.cpp`、`main.cpp`、`md5.cpp` 和 `md5.h`。

`PCFG.h` 中定义了 `segment`、`PT`、`WorkUnit`、`model` 和 `PriorityQueue`。`segment::ordered_values` 保存某类 `segment` 的候选 value；`PT::content` 保存 `segment` 序列；`PT::curr_indices` 保存除最后 `segment` 外已经实例化的 value 下标。

串行生成逻辑主要在 `guessing.cpp`。原始入口 `PriorityQueue::Generate(PT pt)` 会对最后一个 `segment` 的所有 value 做循环拼接；MPI 版本进一步提供 `PriorityQueue::GenerateRange(const WorkUnit&, vector<string>&)`，将最后 `segment` 的 value 区间拆成 `[begin, end)` 后生成。这个循环是本次 GPU 化的目标。

3.2 可并行部分选择

PCFG 生成过程中，CPU 端需要维护优先队列、选择 PT、派生新 PT 并处理概率排序。这些逻辑分支多、数据结构复杂、单次任务控制开销高，不适合直接整体迁移到 GPU。

相比之下，PT 内部最后一个 `segment` 的批量展开具有规则、批量、相互独立的特点。CPU 只需构造固定 `prefix`，收集最后 `segment` 的 `values`，GPU 中每个线程即可生成一个候选口令：

```
output[i] = prefix + values[i]
```

3.3 进阶分支设计

本次进阶部分实际实现了四条路径，其中后三条对应实验指导书中的三个进阶问题。

- **单 PT baseline:** 一次只把一个 PT 的固定 prefix 和最后 segment 的 values 传到 GPU，每个线程生成一个候选口令。该路径用于和最简单的 CPU 串行生成做正确性与性能对比。
- **分支一，多 PT 一次装载:** 延续 MPI 实验中的 PT 层面并行思想，CPU 一次取出多个 PT，把多个 prefix 和所有 value flatten 成一个大 batch。GPU kernel 通过 `item_pt_ids` 找到每个候选属于哪个 PT，从对应 prefix 数组中取前缀，再拼接本线程的 value。
- **分支二，CPU/GPU 重叠:** 使用 `hipStream_t` 和 `hipMemcpyAsync` 构造 stream pipeline。GPU 执行当前 chunk 的 H2D、kernel、D2H 时，CPU 同时打包下一批 PT，减少 CPU 忙等待。代码中记录 `cpu_overlap_ms`，用于观察 CPU 在 GPU 工作期间做了多少打包工作。
- **分支三，按 PT 工作量调度:** 不同 PT 的 values 数量差别很大。实现中设置 `min_gpu_values`、`target_gpu_values` 和 `max_gpu_values`。小 PT 留给 CPU，避免 GPU 启动和拷贝开销得不偿失；中等 PT 合并到接近目标 batch 后送 GPU；过大的 PT 按上限切块，避免单次传输过大导致等待时间过长。

3.4 CPU/GPU 分工

CPU 端负责从 PCFG 优先队列中选择一个或多个 PT，固定除最后 segment 外的 prefix，收集最后 segment 的 value 列表，把多个 PT 的 prefix、value 和归属关系打包为连续数组，调用 HIP wrapper 并对比 CPU/GPU 输出。调度分支中，CPU 还负责根据 PT 的 value 数量决定小 PT 留在 CPU，大 PT 或合并后的 PT chunk 送入 GPU。

GPU 端负责规则的批量拼接：每个线程处理一个 value，根据 `item_pt_ids[idx]` 找到所属 PT，复制该 PT 的 prefix，拼接当前线程对应的 value，并写入定长输出缓冲区和输出长度。每个线程只写自己的输出区间，没有线程间写冲突，因此不需要 atomic 或 shared memory。

3.5 数据打包方式

当前实现采用简单稳定的定长布局：

```
constexpr int PCFG_GPU_MAX_GUESS_LEN = 64;
constexpr int PCFG_GPU_MAX_VALUE_LEN = 32;
```

多 PT 打包时使用如下数组：

```
char* prefixes; // pt_count * MAX_GUESS_LEN
int* prefix_lens; // pt_count
char* values; // item_count * MAX_VALUE_LEN
```

```
int* value_lens; // item_count
int* item_pt_ids; // item_count, each value belongs to which PT
char* output; // item_count * MAX_GUESS_LEN
int* output_lens; // item_count
```

host 端把多个 PT 展平成 item 维度。第 idx 个线程读取 values[idx] 和 item_pt_ids[idx]，再根据 PT id 读取对应 prefix。这样一次 kernel launch 可以覆盖多个 PT 的候选生成。

3.6 GPU kernel 设计

```
__global__ void generate_guesses_multi_pt_kernel(
    const char* prefixes,
    const int* prefix_lens,
    const char* values,
    const int* value_lens,
    const int* item_pt_ids,
    int item_count,
    char* output,
    int* output_lens) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= item_count) return;

    int pt_id = item_pt_ids[idx];
    const char* prefix = prefixes + pt_id * PCFG_GPU_MAX_GUESS_LEN;
    int prefix_len = prefix_lens[pt_id];
    char* out = output + idx * PCFG_GPU_MAX_GUESS_LEN;
    const char* val = values + idx * PCFG_GPU_MAX_VALUE_LEN;
    int vlen = value_lens[idx];

    for (int i = 0; i < prefix_len; i++) out[i] = prefix[i];
    for (int j = 0; j < vlen; j++) out[prefix_len + j] = val[j];

    int len = prefix_len + vlen;
    output_lens[idx] = len;
    if (len < PCFG_GPU_MAX_GUESS_LEN) out[len] = '\0';
}
```

4 实现方法

4.1 代码结构

本次新增或准备的文件如下：

- `src/gpu/pcfg_gpu_generate.h`: GPU wrapper 接口、定长参数、计时结构。
- `src/gpu/pcfg_gpu_generate.hip`: 单 PT wrapper、多 PT flatten kernel、异步 stream pipeline、按工作量调度函数、CPU 串行生成和正确性比较函数。
- `src/gpu/pcfg_gpu_test.cpp`: 独立测试入口，支持 `check`、`bench`、`multi-check`、`multi-bench`、`overlap-check`、`overlap-bench` 和 `schedule`。
- `Makefile`: 提供 `make gpu` 构建入口。
- `scripts/run_gpu_pcfg.sh`: GPU 正确性和性能运行脚本。
- `scripts/run_cpu_baseline.sh`: CPU baseline 运行脚本。
- `results/result_template.csv`: 跑正式数据前的结果表模板。

当前实现不直接改动原 MPI 主程序，避免影响已有训练、优先队列和 MPI 哈希流程。后续如需接入原框架，可在 `GenerateRange` 或 MPI 的批量任务分发处构造多个 `PcfgGpuPtBatch`，再按调度策略调用 `gpu_generate_guesses_multi_pt` 或 `gpu_generate_guesses_multi_pt_overlapped`。

4.2 HIP 接口与错误检查

所有 HIP API 调用使用 `HIP_CHECK` 宏检查错误。kernel launch 后执行：

```
HIP_CHECK(hipGetLastError());
HIP_CHECK(hipDeviceSynchronize());
```

wrapper 在拷贝前检查 `prefix`、`value` 和输出长度是否超过定长缓冲区，避免 device 端越界写入。

4.3 正确性检查

测试程序先用 CPU 串行函数生成 `prefix + value`，再调用 GPU wrapper 生成同一批输出。单 PT、多 PT 和重叠分支保持输出顺序一致，直接逐项比较；调度分支会把小 PT 留给 CPU、大 PT 交给 GPU，因此输出可能分组不同，检查时排序后比较全集。预期检查输出格式为：

```
[check] batch=1024 matched=1024/1024 PASS
```

4.4 计时方法

CPU 串行生成时间使用 `std::chrono::high_resolution_clock`。GPU 端使用 HIP event 分别记录 H2D 拷贝时间、kernel 时间、D2H 拷贝时间和 GPU 总时间。这样可以区分真正的 kernel 计算时间和数据传输开销。

重叠分支额外记录 `cpu_overlap_ms`，表示 GPU stream 处理当前 chunk 时，CPU 在 host 端打包下一 chunk 所消耗的时间。该值不是额外等待时间，而是原本可能被忙等待浪费的 CPU 计算。

5 实验设计

5.1 实验环境

正式实验计划在 AMD AUP Cloud 的 AMD Radeon 8060S 节点上运行。环境信息通过以下命令记录：

```
hipcc --version | tee report/env_8060s.txt
rocm-info | grep -A 30 -E "Agent|Marketing_Name|Device_Type" | head -n 160 | tee
-a report/env_8060s.txt
rocm-smi | tee -a report/env_8060s.txt
```

当前本地工作区未运行正式 GPU 实验，因此本报告不填写运行结果。

5.2 测试规模

计划测试单 PT batch size 为 1K、10K 和 100K。多 PT 分支测试 4 个 PT 同时装载；重叠分支测试多个 PT chunk 的 stream pipeline；调度分支使用 value 数量差异明显的 skewed PT 列表，观察小 PT 留给 CPU、大 PT 送 GPU 的正确性和后续性能。

5.3 结果表模板

branch	PTs	batch	CPU/ms	H2D/ms	kernel/ms	D2H/ms	GPU total/ms	correctness
single PT	1	1K/10K/100K	TODO	TODO	TODO	TODO	TODO	TODO
multi PT	4	TODO	TODO	TODO	TODO	TODO	TODO	TODO
overlap	4	TODO	TODO	TODO	TODO	TODO	TODO	TODO
schedule	mixed	TODO	TODO	TODO	TODO	TODO	TODO	TODO

预期 batch size 较小时，GPU 版本可能受到 kernel 启动和数据传输开销影响，加速不明显甚至出现负加速。随着 batch size 增大，GPU 并行线程数量增加，kernel 计算占比上升，H2D/D2H 开销被更多候选口令摊薄，GPU 版本才更可能体现优势。多 PT 分支

预期可以提高 GPU 利用率；重叠分支预期减少 CPU 忙等待；调度分支预期避免小 PT 负加速。该段仅为实验前分析，不作为实验结论。

6 跑数据后补充的实验结果与分析

TODO: 在 AUP AMD GPU 环境运行 `scripts/run_gpu_pcfg.sh`, 确认 correctness 为 PASS 后填写真实性能结果、speedup 和分析。

7 总结

基础部分总结了 6 个 HIP Notebook 的学习过程，覆盖线程索引、二维矩阵映射、atomic、shared memory、reduction、Monte Carlo 采样和 K-Means 迭代并行。进阶部分选择 PCFG 口令猜测中 PT 内部最后 segment 批量展开作为 GPU 化对象，保留 CPU 端复杂调度逻辑，将规则且独立的候选生成循环迁移到 HIP kernel，并完成多 PT 一次装载、CPU/GPU stream 重叠和按 PT 工作量调度三个分支。当前代码、脚本和结果模板已准备好，正式性能数据需要在 AMD AUP GPU 环境中运行后补充。